

DAY 2 · SPEC-DRIVEN DEVELOPMENT

Agentic Software Development

Day 2: building with AI responsibly

Tino Silfver · Jukka Loikkanen

Stack: Java 25, Spring Boot 3.5



Prerequisites for the labs

- JDK 25 (java --version), Git, a GitHub account with your Copilot licence (SDKMAN should help)
- VS Code with GitHub Copilot, Copilot Chat, and the Java extension pack
- The lab repo cloned, and the build run once so it is green
- No Node, no npm, no database, no Docker (that is a feature)

◆ RUN CHECK-SETUP

If unsure, run the check-setup script and resolve any FIX lines before we start. The first dependency download is the one thing that bites a big room: it should already be done on your own network.

Today's flow

- Morning: the destination, the landscape, AI-assisted development foundations, spec-driven development
- Afternoon: one continuous project, specification to documentation
- Closing: agent workflows and the methodologies that formalise them

Situation check

- Where are we now with AI in development?
- Where do we want to be?
- What is blocking us today?



One ticket, five agents, one verdict

- A ticket goes in: one feature request, with a compliance check built in
- Five agents handle it in sequence: Spec-Plan, Code, Doc, Verdict, orchestrated
- A COMPLETE verdict comes out
- Every stage has a human review gate

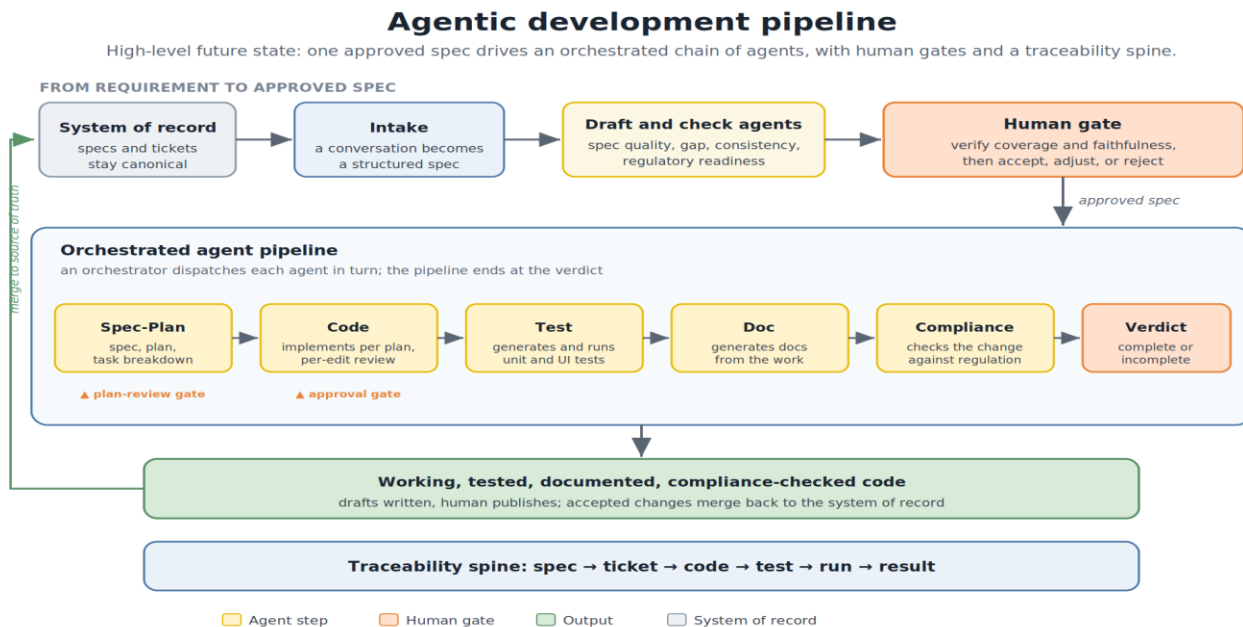
◆ WATCH IT RUN

We start by showing the finished machine. This afternoon you build each stage yourselves. By 15:00 you understand everything you just watched.

What an agentic development pipeline looks like

- A spec drives everything: the code that implements it and the tests that verify it
- Agents draft and check; a human gate accepts, adjusts, or rejects
- Documentation is generated from the work, not written after as a chore
- Structure is what makes this possible: agents read structure, not prose
- Every artifact links back to the spec that produced it (traceability)

The future-state agentic development pipeline



Structured documentation for the pipeline

Four design principles

Domain, not phase

Domains are stable for the life of the solution; phases end and are metadata only

One canonical home

Every fact lives in exactly one place; everything else links to it

Deterministic for machines

Numbered sections; one page equals one Markdown file

Small pages, sized for AI

300-800 words per page; agents navigate then fetch, not hope then fetch

Confluence top-level folders (example)

- 00 PROGRAM
- 01 SDD (cross-cutting, new)
- 10 ARCHITECTURE
- 20 CUSTOMER PROJECT 1
- 30 INTEGRATIONS
- 40 CUSTOMER PROJECT 2

Per-domain skeleton (example)

- 0 Overview and scope
- 1 Specifications
- 2 Interfaces
- 3 Test basis
- 4 Decisions
- 5 Operations
- 9 Archive (excluded from agent grounding)

The landscape in one picture

- Models, platform, agents, governance: how the layers stack
- Microsoft and Anthropic ecosystems side by side
- Where today's work (Copilot plus spec-driven development) sits

Azure OpenAI and the Landing Zone

- Models hosted inside your Azure tenant
- Part of Topcon's Azure Landing Zone (in progress)
- Data and governance stay within your perimeter

◆ WHY IT MATTERS

The perimeter is the point in healthcare: a managed environment is what makes the two days responsible by default. Connect to the Landing Zone work; mention only briefly.

Agent 365 and the Azure SRE Agent

- Agent 365: observe, govern and secure agents as managed identities (works best with Microsoft E5)
- SRE Agent: AI-assisted incident response for future maintenance
- The loop: SRE Agent opens a GitHub issue, Copilot proposes the fix

What this means for today

Today

- Build: Copilot and spec-driven development
- Run: SRE Agent and operations (tomorrow's win)
- Govern: Agent 365 across the lifecycle

Your decisions now

- Position you for all three
- A Microsoft stack leans on SRE Agent and Agent 365
- An Anthropic stack looks different

How we got here

- From autocomplete to chat to agents
- The constant: a human who understands the code
- The variable: how much the tool does per step

GitHub Copilot in VS Code

- Inline completion, Copilot Chat, agent modes
- Choosing the right mode for the task
- The model behind Copilot is a choice, not a constant
- Today's driver for the whole project

AI models are not interchangeable

- The model behind Copilot is a choice. Different models, different results.
- Quality, speed, context window and cost all vary by model
- Know which model you are using; try another before blaming the tool
- Model availability is now a compliance and geopolitical variable too

◆ RE-VERIFY ON THE DAY

June 2026: an export-control directive disabled the two most capable models for all customers. Our tooling (Copilot on Sonnet 4.6 or Opus 4.8) is not affected. Availability is now a geopolitical variable, not only a technical one.

Context: what Copilot sees

- Open files, selection, workspace signals
- What it cannot see, and why that matters
- Steer context deliberately rather than hoping
- Keep the codebase index current; a stale index gives weaker answers

Token usage and the context window

- The model has a fixed context window; everything competes for it
- Instructions, tool definitions, open files, prior messages, tool results
- More in the window is not better: it crowds out what matters and costs tokens
- Habits: keep relevant files open, start fresh, compact long threads

◆ AGENTS PAY TO READ

Well-structured docs cost fewer tokens, so quality and cost pull the same way. This ties straight to Topcon's documentation reality (slide 28).

Copilot is configurable, beyond the conventions file

- Instructions: always-on rules (the conventions file we use today)
- Prompts: reusable saved prompts. Skills: instructions and resources loaded when relevant
- Agents: custom personas (built-in Ask and Plan, plus custom in `.github/agents`)
- Hooks: lifecycle actions. Plugins and MCP Servers: external tools

Prompting for code

- State intent, constraints and examples
- Small, verifiable steps beat large unchecked blocks
- Iterate: review, correct, continue

You are the reviewer

- Hallucinated APIs and packages happen
- Copy-paste-commit without review is the cardinal sin
- Unattended agent loops need oversight
- "Copilot did it" is not a review

◆ THE ABSOLUTE

Ownership is the absolute; disclosure is contextual. You own the code regardless of how it was drafted. Disclose AI help where trust depends on it, in the PR or commit body, not the authorship trailer.

Agent modes: Ask, Plan, Agent

The three modes

- Ask: read-only exploration, no changes
- Plan: produce an implementation plan from a spec
- Agent: the supervised, multi-step build

They map onto

- Spec, Plan, Code
- Use the right mode for each task
- Default Approvals on, not Autopilot, so the gate stays real

Copilot Enterprise, briefly

- Repo-level context, PR automation, governance features
- Requires the Enterprise tier
- Worth knowing what the next step offers

Copilot subagents: parallel runs and /fleet

What /fleet does

- ▶ A parallel task dispatcher in the Copilot CLI
- ▶ Decomposes a large prompt into isolated subtasks
- ▶ Launches independent, stateless subagents simultaneously
- ▶ Today's custom agents run sequentially (our pipeline). /fleet runs tasks in parallel, for large multi-file changes.

How subagents evolved

Oct 2025 Subagents in VS Code Insiders (context-isolated tasks)

Jan-Feb 2026 Specialized custom agents and orchestration

Mar-Apr 2026 Recursive subagents; parallel task execution

Jun 2026 /fleet widely available in Copilot CLI

The demo setup, shown not just described

- The portal UI in its starting state: an empty page that comes alive as APIs are built
- The structure: src/, docs/specs/, docs/compliance-requirements.md, legacy/, .github/
- The conventions file is the quiet authority behind every generation
- The .github/agents folder is where the morning's pipeline agents live

Demos, Module 3

1

Demo 3.1

Generate code from requirements
(production code only)

2

Demo 3.2

Refactor legacy code, 2006 to 2026

3

Demo 3.3

Generate unit tests (the
conventions wanted them all
along)

Why requirements matter

- AI amplifies whatever you feed it
- Vague intent becomes confident, wrong code
- The fix is upstream, not in the prompt

The specification is the source of truth

- Write the spec first, generate against it
- Code becomes an output; the spec is the maintained artifact
- Reviewable, repeatable, traceable

Spec, Plan, Tasks, Code

1

Spec

What and why

2

Plan

How, in checkable
steps

3

Tasks

Units small enough to
verify

4

Code

Generated, reviewed,
tested

AI-ready documentation

- Test-ready: clear enough to generate tests from
- Operating: usable at run and maintenance time
- Structured for retrieval: agents read structure, not vibes

Topcon's documentation reality

- Control documents in SharePoint
- Technical documents in Confluence
- A split that is hard for agents to traverse
- Structure drives token cost: agents pay to read

◆ THE FIX

This is the messy starting point the future-state pipeline (slide 7) is built to fix: a one-time transformation sweep, then steady-state maintenance.

Traceability is audit-ready

- A written spec and decision trail beats commit history alone
- In a regulated context this is a strength, not overhead
- The same habit spec-driven development already gives you

Demos, Module 4: one feature, three ways

The runs

- 4.1 good spec plus conventions: correct, seeded (8081)
- 4.2 poor prompt plus conventions: drifts, empty (8082)
- 4.3 poor prompt, no conventions: breaks loudly (8083)

The lesson

- The screen is the judge
- One variable changes per step
- The model did not get smarter between tabs; only the briefing changed

This afternoon, one project, end to end

- Service Operations Portal: assets, maintenance records, work orders, service notes
- Lightweight web app, in-memory data, VS Code plus GitHub Copilot
- Labs 3 to 7: specify, plan, implement, extend with a regulatory check, document, design the workflow
- Everyone has a role: spec writing is not coding; implementation runs in pairs

Lab 3, create the specification

- Turn a requirement into a spec, the Spec-Plan agent's first job
- Start from the prepared skeleton spec and conventions
- Refine: user story, acceptance criteria, non-functional requirements, test scenarios
- Output: an approved feature specification

◆ HANDOFF

Save as docs/specs/spec-lab.md. Use Ask mode to explore read-only first. Spec writing is not coding: non-technical participants take full part. Lab 4 reads this file.

What good looks like (spec checklist)

- Testable acceptance criteria, no ambiguity an agent could misread
- Non-functional requirements stated, not implied
- Out-of-scope named explicitly: the things you name out of scope are what the poor-prompt demo invented

Lab 4, plan the implementation

- Turn the spec into a plan, the Spec-Plan agent's second job
- Feed the approved spec to Copilot, in Plan mode
- Produce: implementation plan, task breakdown, test strategy
- Review the plan critically and change at least one step

◆ STARTER PROMPT

Read docs/specs/spec-lab.md and produce an implementation plan at docs/plans/plan-lab.md. Ordered steps, each verifiable, a test strategy mapped to acceptance criteria. Do not write code.

Lab 5, implement with GitHub Copilot

- Implement the plan in Agent mode, with a review gate at each edit
- Domain model, API, tests, refactor, from the plan
- Work in pairs: driver and navigator
- Small steps, review each before the next

◆ DEFAULT APPROVALS

Every file write asks you to approve before it applies. That is the review gate. Confirm it is on, not Autopilot. Starter: implement step 1, run the tests, wait for approval before step 2.

The regulatory-check extension

- Satisfy a written compliance requirement: REG-01, an append-only audit trail
- Pull the compliance check out into its own agent (a compliance-reviewer)
- The trail from spec to code must show the requirement was satisfied
- MDR and HIPAA, as a training simulation, not legal advice

◆ GROW THE PIPELINE

In the demo, compliance lived inside Verdict. Here you split it into its own agent, because a regulated system wants compliance as an explicit, separately-owned gate. Devices, never patients.

Lab 6, documentation for AI

- Generate operational and developer documentation from the feature
- Structure it for AI retrieval
- Update the changelog; update any doc the change touched
- Output: an AI-ready documentation package

◆ OPTIONAL BEAT

Hand the doc run to a background agent, keep talking, then review the output together. That review is the human approval gate, and it seeds the Lab 7 canvas.

Lab 7, agentic development workshop

- You just built, by hand, what the pipeline did at 09:20
- In small groups, sketch the agent workflow for this project
- Human approval points, governance model, documentation requirements
- Output: an agent architecture canvas

You just invented a methodology

- Your canvas: roles, approval gates, documents
- The industry has formalised exactly this
- Three worth knowing, plus the wider field

GitHub Spec Kit

- Open-source spec-driven toolkit: Spec, Plan, Tasks, Implement
- Fits the VS Code and GitHub Copilot world
- A project "Constitution" can encode security and compliance rules

OpenSpec

- Lightweight, tool-agnostic, open source
- A living source-of-truth spec
- Proposal, apply, archive flow

The joint Black Belt and AO Finland methodology

- A structured agentic model
- Role-divided agents and decision documents
- Built for traceability and regulated contexts

◆ THE FULL METHOD

This is the same shape you saw this morning, formalised. The pipeline reveal and the future-state diagram were the simplified, teachable version of this.

Others in the space

- BMAD-METHOD: a virtual agent team
- Amazon Kiro: AWS-native (a contrast, not a fit here)
- Tessl: spec as the maintained source

A decision path for Topcon

- Start with documentation and a simple spec habit
- Pick one tool track and one pilot project
- Let the process change unlock the rest

◆ FROM HERE

Bridge back to the morning's situation check: from where we are to where we want to be, and the future-state diagram is the want-to-be. Black Belt offers rollout help with very similar tooling.

DAY 2 · WRAP-UP

Kiitos.



You saw the finished pipeline at 09:20 and built a working slice of it by 15:00.

What you built today: requirements to working, documented, compliance-checked code

Open Q&A. Feedback and certificates. Cheat sheets to take home.

